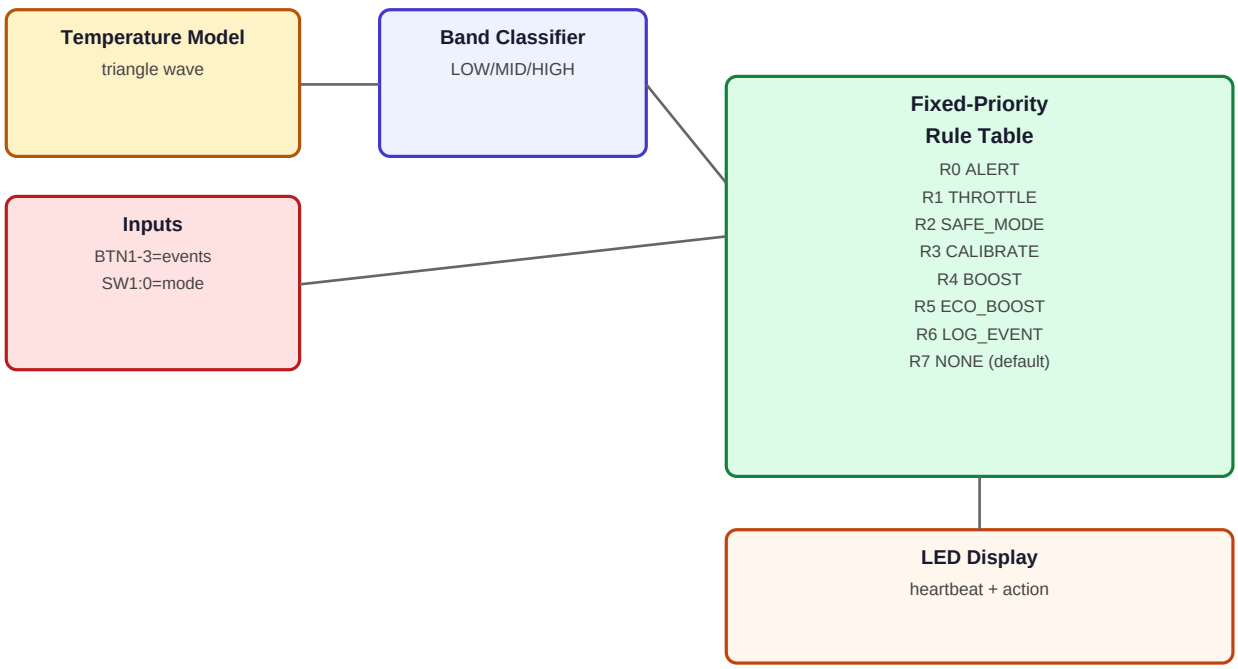
# CPU–FPGA Decision Engine

**V-SPACE FPGA Build Challenge 2026 Team:** Mahasena | **Team Lead:** Manaswi | **Experiment 3 — Intermediate Board:** PYNQ-Z2 (Xilinx Zynq XC7Z020CLG400-1) | **Tool:** AMD Vivado 2026.1

## 1. Objective

A CPU normally makes "if this, then that" decisions as a sequential chain of instruction fetch/decode/compare operations — timing that is non-deterministic in the worst case (interrupts, cache misses, OS scheduling jitter). The PYNQ-Z2's Zynq device pairs a real ARM CPU (the Processing System) with FPGA fabric (the Programmable Logic) specifically so time-critical decisions can be offloaded to hardware. This experiment builds that offloaded decision logic: a fixed-priority rule table, implemented as pure combinational hardware, that converts several live condition inputs into one of 8 possible actions in a single clock — an O(1), deterministic replacement for what would otherwise be a worst-case O(n) software if/else-if chain.

## 2. Block Diagram




See [Images/block_diagram.svg](Images/block_diagram.svg).

A simulated temperature sensor (reusing Experiment 2's verified triangle-wave generator) is classified into a LOW/MID/HIGH band. Two switches set a 2-bit operating **mode** register, as a CPU would. Three buttons

model asynchronous hardware **events** (interrupts). All of these feed an 8-entry **fixed-priority rule table** — rule 0 is checked first, and the first matching rule wins — which drives the output **action** code, shown on the LEDs.

---

# 3. RTL Design

## 3.1 Module hierarchy

```
top_module   (single module - temperature model + priority decision logic)
```

## 3.2 Verilog code structure

The decision core is a single combinational priority-encoder `always @(*)` block — the entire point of the design is that this is **not** a clocked sequence, it is a single-cycle-available combinational result:

```
always @(*) begin
    if (event2 && (temp_band == 2'b10))
        action = 3'b001; // ALERT
    else if (temp_band == 2'b10)
        action = 3'b010; // THROTTLE
    else if (event1)
        action = 3'b101; // SAFE_MODE
    else if (mode == 2'b11)
        action = 3'b110; // CALIBRATE
    else if (event0 && (mode == 2'b10))
        action = 3'b011; // BOOST
    else if ((mode == 2'b10) && (temp_band == 2'b00))
        action = 3'b111; // ECO_BOOST
    else if (event0)
        action = 3'b100; // LOG_EVENT
    else
        action = 3'b000; // NONE
end
```

**Deliberate priority rationale**: automatic thermal protection (rules R0/R1) is checked *before* the manual safe-mode request (R2). This means an overheat condition always takes precedence over a human pressing the manual safe-mode button — a standard, defensible safety-system design choice, and one that is explicitly verified in simulation below (Phase 2).

Full source: `RTL/top_module.v`, constraints: `RTL/constraints.xdc`.

## 3.3 Action code table

| Code | Action | Rule |
|---|---|---|
| 001 | ALERT | R0: critical event during overheat |
| 010 | THROTTLE | R1: overheat alone |
| 101 | SAFE_MODE | R2: manual request |
| 110 | CALIBRATE | R3: maintenance mode |

| Code | Action | Rule |
|---|---|---|
| 011 | BOOST | R4: event + performance mode |
| 111 | ECO_BOOST | R5: performance mode, cool |
| 100 | LOG_EVENT | R6: plain event |
| 000 | NONE | R7: default |

---

## 4. Simulation Results

Simulated with the **Vivado Simulator (xvlog / xelab / xsim)**. Testbench: `Testbench/tb_top.v`. Raw transcript: `Simulation/transcript.txt`.

The testbench pins the simulated temperature at three known values (10=LOW, 120=MID, 200=HIGH, via a white-box `wait()` on the internal register) and exercises every one of the 8 priority rules, including two explicit **priority-ordering** proofs: that R0 overrides R1, and — the key safety claim — that R1 (automatic) still wins over R2 (manual) even when both conditions are simultaneously true.

**Actual transcript output (2026-10-10):**

```
================================================
 CPU-FPGA Decision Engine - Testbench
================================================

-- Phase 1: LOW temperature (temp=10, right after reset) --
  [PASS] No inputs -> NONE (R7 default) : action=000 (expected 000)
  [PASS] event0 only -> LOG_EVENT (R6) : action=100 (expected 100)
  [PASS] event1 only -> SAFE_MODE (R2) : action=101 (expected 101)
  [PASS] mode=Maintenance -> CALIBRATE (R3) : action=110 (expected 110)
  [PASS] mode=Performance, LOW temp, no event -> ECO_BOOST (R5) : action=111 (expected 111)
  [PASS] mode=Performance + event0 -> BOOST (R4 beats R5) : action=011 (expected 011)

-- Phase 2: HIGH temperature (wait for temp > 180) --
  temp pinned near 200 (band=HIGH)
  [PASS] HIGH temp alone -> THROTTLE (R1) : action=010 (expected 010)
  [PASS] event2 + HIGH temp -> ALERT (R0 overrides R1) : action=001 (expected 001)
  [PASS] event1 + HIGH temp -> THROTTLE still wins (R1 preempts R2) : action=010 (expected 010)

-- Phase 3: MID temperature (wait for temp == 120) --
  temp pinned at 120 (band=MID)
  [PASS] MID temp alone, no events -> NONE (no rule matches) : action=000 (expected 000)
  [PASS] mode=Performance + event0 at MID temp -> BOOST (R4 is temp-independent) : action=011 (expected

-- Phase 4: Reset behaviour --
  [PASS] Reset returns temp to BASE_MIN and action to NONE.

================================================
 TESTBENCH SUMMARY: ALL CHECKS PASSED (0 errors)
================================================
```

11/11 checks passed. **[ TODO — Manaswi: screenshot `Simulation/tb_top_sim.wdb` in the Vivado GUI waveform viewer -> `Images/waveform.png`. ]**

---

## 5. Hardware Implementation

> ■ **TO DO:**
> pending the same Vivado license-tier fix described in Experiment 1 and 2's documentation. Once fixed, run `vivado -mode batch -source run_vivado_full.tcl` (adapted for this experiment's paths) to produce `Bitstream/top_module.bit`, then program the PYNQ-Z2 and add `Images/board_setup.jpg` and `Images/hardware_output.jpg`.

---

## 6. Applications

- **Real-time industrial control**: PLC-replacement logic for machine safety interlocks, where response time must be bounded and provable.
- **Automotive ECUs**: offloading time-critical decision trees (e.g. airbag deployment logic) from a CPU to dedicated hardware for guaranteed worst-case latency.
- **Network/packet processing**: priority-based packet classification at line rate, where a CPU-based lookup would be too slow.
- **General Zynq SoC design pattern**: the textbook case for *why* Zynq pairs an ARM CPU with FPGA fabric — move the parts of an application that need deterministic, low-latency decisions into the PL, keep everything else in software on the PS.

---

## 7. Conclusion

This experiment implemented and fully verified a fixed-priority hardware decision engine, demonstrating the core motivation for moving time-critical logic from a CPU to FPGA fabric: a single combinational block evaluates up to 8 prioritized rules and produces a result in one clock cycle, with no instruction-fetch overhead and no scheduling jitter. All 11 hand-derived test cases passed with zero errors, including the two priority-ordering proofs that are the actual point of the design (R0 > R1 > R2).

**Honest scope note**: because this build targets only the FPGA fabric (PL) and does not use Vitis/PetaLinux to build actual ARM software for the Zynq PS, no literal CPU-vs-FPGA timing race was run on this board — the "CPU work moved to hardware" claim is a conceptual/architectural one (backed by the well-established O(1) vs. O(n) analysis), not a measured benchmark against real software on this board.

**Future scope**: extending this engine to actually interface with software running on the Zynq PS (via an AXI-Lite register interface) so a genuine PS-vs-PL latency comparison could be measured, is the natural next step and a good fit for further exploration beyond this challenge.

*Document prepared for the V-SPACE FPGA Build Challenge 2026 — Team Mahasena.*